

COMP2001: ARTIFICIAL INTELLIGENCE

METHODS – LAB 5 EXERCISES

RECOMMENDED TIMESCALES

This lab exercise is designed to take no longer than two hours, and it is recommended that you complete this within a week. The contents of this exercise were covered in lecture 5.

GETTING SUPPORT

The computing exercises have been designed to facilitate flexible studying and can be worked through in your own time or during timetabled lab hours. It is recommended that you attend the labs, even if you have completed the exercises in your own time, to discuss your solutions and understanding with one of the lab support team and/or your peers. It is entirely possible that you might make a mistake in your solution which leads to a false observation and understanding.

OBJECTIVES

The purpose of this lab exercise is to gain experience in implementing a population-based method that was covered in lecture 5 and performing some statistical analysis of multiple (>2) algorithms.

The topics that will be covered in this exercise are:

- COMP2001 framework background for handling populations of solutions.
- Implementation of a memetic algorithm.
- Empirical evaluation of local search in memetic algorithms.
- Statistical analysis of multiple comparisons.

LEARNING OUTCOMES

By completing this lab exercise, you should meet the following learning outcomes:

- Students should gain experience in implementing (multi point/population based) metaheuristics for solving combinatorial optimisation problems.
- Students should understand and be able to implement components of evolutionary algorithms including crossover, mutation, local search, parent selection, and population replacement.
- Students should be able to use statistical tests that goes beyond comparing two algorithms under the same conditions.

TASK 1 – IMPORTING LAB EXERCISE FILES

Download the source code for this lab exercise from the Moodle page. You should find various files related to genetic and memetic algorithms, the exercise runner configurations, and the exercise runners themselves.

Drag the **population** folder into your IDE so that it is a direct subfolder of `com.aim.metaheuristics`.

The `Lab5ExercisesTestFrameConfig` and `Lab5ExercisesRunner` classes should be placed within the `runner` package alongside the ones from previous labs.

TASK 2 – READING: FRAMEWORK BACKGROUND

A memetic algorithm is one example of a population-based search method. The COMP2001 Framework allows multiple solutions to be stored in memory, the difference compared to single point methods is that we have 32 memory addresses to manage for a population size of 16. 16 addresses for the current population of solutions, and another 16 for the offspring population. Typically, we do not require backup solutions hence the 16+16 memory indices are sufficient. Special methods are needed for applying heuristics to the correct solutions in memory and these are given in the **implementation hints** section.

Below is an illustration of a population-based search method using the COMP2001 Framework. Notice that at the start of each iteration (generation), the current population should be stored in the first `POPULATION_SIZE` indices of the solution memory. The second `POPULATION_SIZE` indices should be used for creating and mutating/improving the offspring in each generation. At the end of each iteration, `POPULATION_SIZE` solutions should be selected (from either parent or offspring populations) and moved to the first `POPULATION_SIZE` indices according to the replacement scheme.

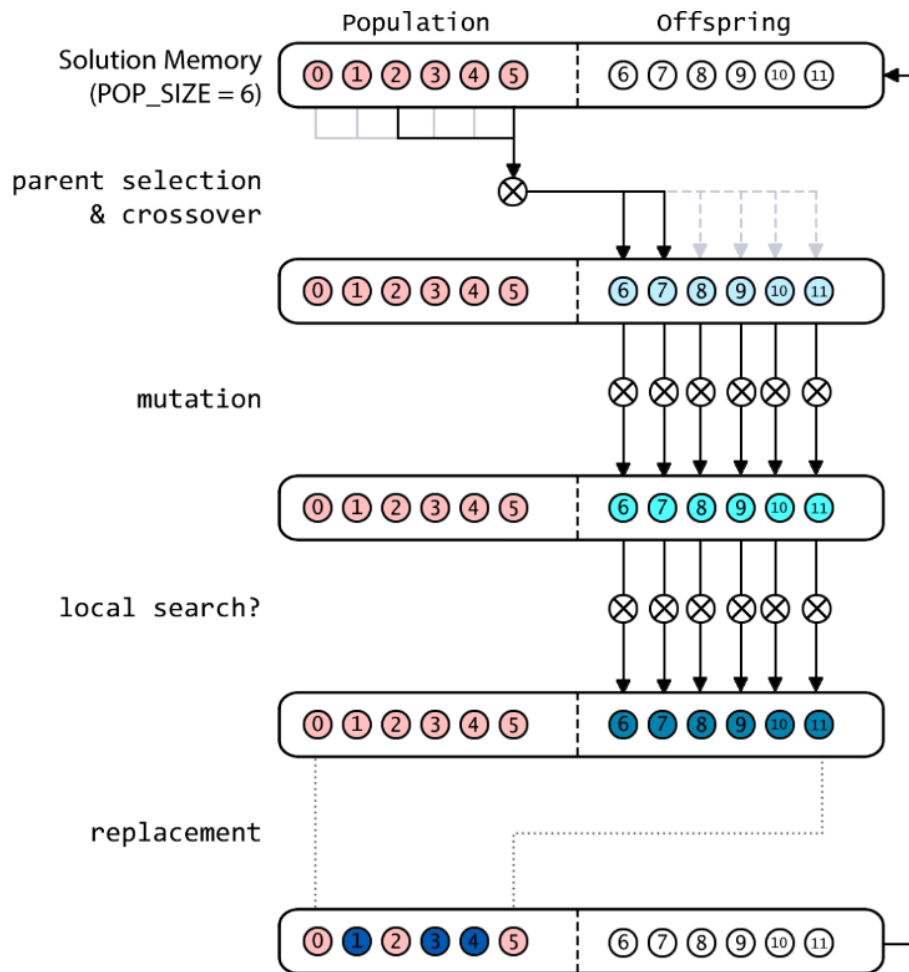


Figure 1 – Population based Search illustrating a Memetic Algorithm in the COMP2001 Framework. Note that both the current population and the offspring share the same contiguous memory with indices [0..POPULATION_SIZE-1] storing the current population, and indices [POPULATION_SIZE..POPULATION_SIZE*2-1] storing the offspring (children).

Rather than having number of evaluations as the termination criterion, it is common for the termination criterion of evolutionary methods to be defined as the maximum number of generations. For the Memetic Algorithm, we therefore use number of generations as the termination criterion rather than a “time limit”/evaluation count as we did in all previous lab exercises.

TASK 3 – MEMETIC ALGORITHMS [IMPLEMENTATION]

Memetic Algorithms (MA's) were covered in lecture 5. Below is the pseudocode for a **Genetic Algorithm**. The difference between GA's and MA's is the inclusion of a local search step. It is entirely up to you to decide which place (or places) to apply local search – you will be finding a suitable placement in one of the formative quiz exercises.

Like the previous labs, you only need to implement the main loop of the GA (lines 3-7). The outermost loop and initialisation are handled by `Lab5ExerciseRunner.java`.

Samples of each different component are implemented for your reference, and you will find these as additional source files in the code you downloaded. Your task is to implement the components below and run the experimentation as outlined in the lab 5 formative assessment.

MEMETIC ALGORITHM

Below is the pseudocode for a **Genetic Algorithm**. You should implement this and include a local search step to form a Memetic Algorithm, but it is up to you to decide where this should be. You can run a series of experiments to determine the best location (see the quiz exercises). Once you have implemented the memetic algorithm, remember to implement the other genetic operators outlined in the following sections.

Note that when selecting two parents, it may be the case that the parent selection mechanisms have chosen the same solution for both parents. In the even that this happens, you should **choose the second parent to be the solution in the next index along** (be careful with the two parent/offspring populations!).

```
s[0-N) = generateInitialPopulation()    // handled by the lab exercise runner.
REPEAT max_generation TIMES:           // handled by the lab exercise runner.
    REPEAT population_size / 2 TIMES:
        (p1, p2) <- Select two unique parents
        (c1, c2) <- crossover(p1, p2)
        Apply mutation to children (c1, c2)
    DO population_replacement
return s*                               // handled by the lab exercise runner.
```

TOURNAMENT SELECTION

Random Selection and **Fittest Selection** have been implemented for you, and you can use these as a basis for your implementation of tournament selection. In **Tournament Selection**, a parameter, `tournament_size`, is used to determine the size of a subset of parent solutions that are to be selected at random. The best solution is then chosen from this subset of parent solutions and its index returned for the selection procedure. If the best solution is tied with at least one other solution, then a solution is chosen randomly from the subset of best solutions from the subset of tournament selected solutions.

For example, given the following parent solutions of population size 8, and a tournament size of two selecting at random solutions [0] and [2], the chosen solution would be [2] since the objective value of 88 is the best in the subset { 88, 100 }.

Index	0	1	2	3	4	5	6	7
Cost	100	89	88	91	120	80	92	64
Selected			✓					

```
1 | INPUT: parent_pop, tournament_size
2 | solutions = getUniqueRandomSolutions(tournament_size);
```

```

3 | bestSolution = getBestSolution(solutions);
4 | index = indexOf(bestSolution);
5 | return index;

```

UNIFORM CROSSOVER

An example of **1-point Crossover (1PTX)** has been implemented for you to study. Crossover operators are generally used to create offspring from two parent solutions. There are `POPULATION_SIZE` parent solutions in the current population, and space for `POPULATION_SIZE` offspring solutions in the next `POPULATION_SIZE` memory indices (see Figure 1).

In Uniform Crossover (UXO), the idea is to copy the two parents (p1 and p2) into two offspring indices (c1 and c2) whereby each bit in the bitstring is exchanged between the offspring with a probability of 50%.

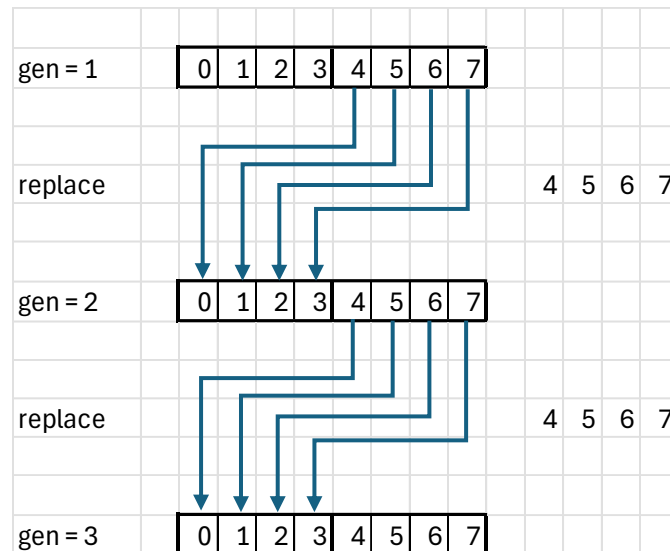
```

INPUTS: p1, p2, c1, c2
memory[c1] = copyOf(p1)
memory[c2] = copyOf(p2)
FOR j ∈ [0, chromosome_length):
    IF random ∈ [0,1) < 0.5 THEN
        exchange(c1,c2, j)

```

TRANS-GENERATIONAL REPLACEMENT WITH ELITIST REPLACEMENT

A simple replacement strategy replacing the parent population in the next generation with all the offspring population in the current generation is implemented in **BasicReplacement**. In the COMP2001 framework, between each generation, the current parent population is replaced by a subset of the union of the current parent and offspring populations by specifying the indices of the solutions in memory that we wish to carry forward. In a basic replacement strategy, we wish to set the current population in the next generation as the offspring population in the current generation. For example, if the population size was 4 and we wanted to carry forward all offspring solutions, then we would preserve $s \in [4,5,6,7]$ by specifying an integer array containing 4, 5, 6, and 7.



The Trans-Generational Replacement with Elitist Replacement scheme should be implemented in **TransGenerationalReplacementWithElitistReplacement** and is very similar to the basic replacement method but with one key difference; if the offspring population does not contain the best solution of all solutions in the parent and offspring populations, then the best solution from the parent population should replace the worst solution in the offspring population (see the pseudocode and diagram below).

```

1 | INPUT: current_pop, offspring_pop
2 | fitnesses <- evaluate( current_pop U offspring_pop );
3 | best <- min(fitnesses);
4 | next_pop <- indicesOf( offspring_pop );
5 | IF best ∉ offspring_pop THEN
6 |     next_pop.replace( worst, best );
7 | ENDIF
8 | OUTPUT: next_pop; // return the indices of the next population

```

Below is an illustration where the best solution resides first in the offspring population, and then in the current population demonstrating which solutions are carried over into the next generation.

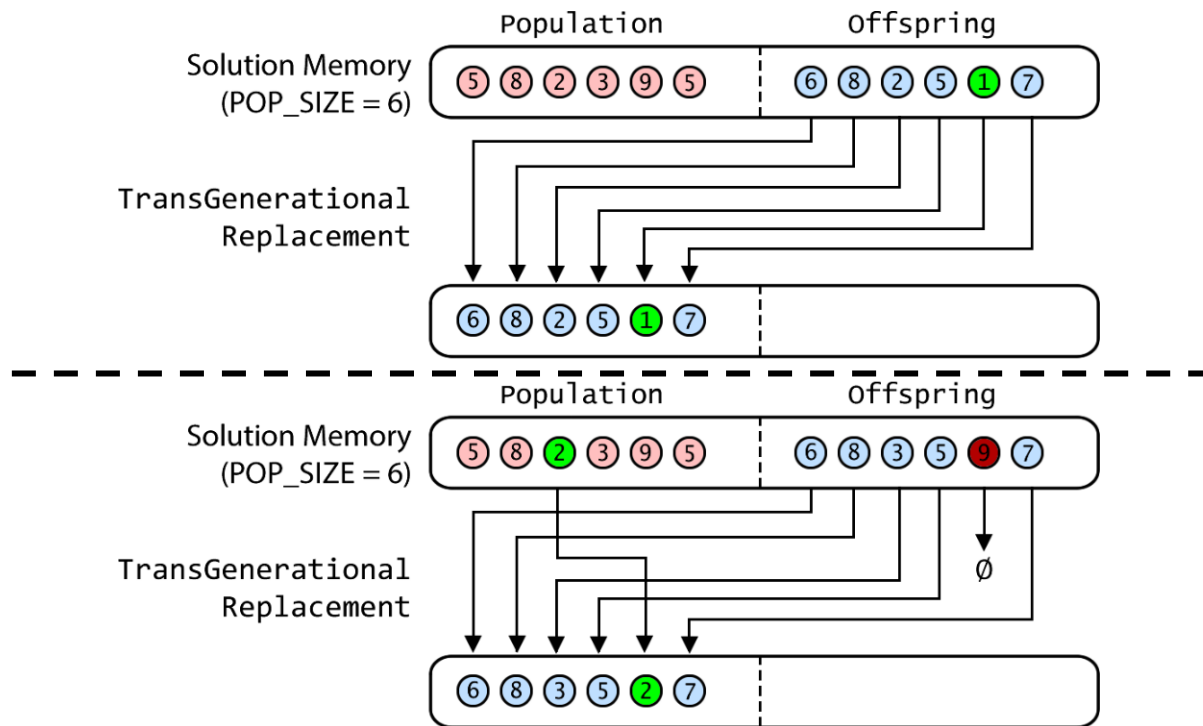


Figure 2 - Illustration of Trans-Generational Replacement with elitist best replacement. The circles represent solutions in memory and the numbers contained in them represent their objective values. TransGenerationalReplacementWithElitist-Replacement will return [6,7,8,9,10,11] in the first example and [6,7,8,9,2,11] in the second.

TASK 4 – FORMATIVE ASSESSMENT (MOODLE QUIZ)

The questions in the “[Lab 5 Formative Assessment](#)” require you to perform a number of experiments using your implementation of the memetic algorithm and its components. Answer the questions by configuring the `Lab5ExercisesRunner` and `Lab5TestFrameConfig` classes as appropriate, running the algorithms, and analysing the resulting outputs. You may want to keep a copy of the results presented after running each of those experiments; instructions are given at the end of this document.

Note that open text box questions will receive general feedback within a week of the formative quiz opening. If you would like additional feedback on your answers, please come and speak to one of us in the lab!

ADDITIONAL INFORMATION AND TIPS

How do I perform a bit flip on a specific solution in memory?

`problem.bitFlip(i, solutionIndex)` is used to flip the i^{th} bit of the solution in memory index `solutionIndex`.

How do I swap/exchange the values of a specific bit of two solutions?

`problem.exchangeBits(a, b, i)` is used to exchange the i^{th} bit between solutions in memory indices a and b .

How do I apply the various heuristics within the MA?

Mutation and local search operators are both `PopulationHeuristic`'s and are invoked by calling the `applyHeuristic(int index)` method where `index` is the index in solution memory of the solution to apply the heuristic to.

Crossover operators are invoked by calling the `applyHeuristic(p1, p2, c1, c2)` method where `p1` is the memory index of the first parent, `p2` is the memory index of the second parent, `c1` is the memory index of the first child, and `c2` is the memory index of the second child.

The replacement scheme is performed by invoking the `doReplacement(problem, POP_SIZE)` method.

How do I run the algorithm in GA/MA mode?

There is a configuration in the `Exercise3aTestFrameConfig` class which allows you to run your algorithm in MA mode or GA mode. The framework replaces the hill climbing heuristic with a "NOOP" when in GA mode.

How do I: perform a bit flip; get the number of variables; evaluate the solution; etc.

See the COMP2001 Framework API for framework specific questions on Moodle!